

Extracting business rules from information systems

J Shao and C J Pound

Business rules are an important asset of an organisation. Many organisations have substantial investment in business rules encoded in legacy information systems. This paper reports on the types of business rules that may be found in an organisation's information systems, and the techniques that may be used to recover business rules from such systems. Also outlined is a new approach which may be used to unlock business rules from legacy information systems.

1. Introduction

Business rules are nothing new. Every organisation expresses and enforces its business rules one way or another. Business rules represent core business practices and policies, and they determine what is possible or desirable in running a business [1, 2]. For example, the following is one of the business rules that an organisation may have:

‘Customers who have not placed an order within the last two years are marked inactive at the end of each month.’

It should be noted that an organisation typically has many business rules, and some of the rules are related. For example, there may be another business rule stating that ‘customers with uncleared debts must remain active’. The related business rules must be read together to understand a particular business practice or policy.

The business rules of an organisation may be expressed and enforced in different ways. Some may be documented formally as the organisation's policies, while others may be built into the operational procedures of the organisation. However, the increased use of computers has resulted, over the years, in many business rules that are now enforced by the supporting information systems. For example, if the customer information is stored in a database, the above business rule may be implemented as a database trigger, as shown below (the syntax used here is for illustration only and does not correspond to any actual implementation):

EVENT	end of month
CONDITION	customer c has not placed an order within the last 2 years
ACTION	mark customer c inactive

Clearly, an organisation's ability to understand, manage and evolve its business rules within its information systems

is crucial to its operational effectiveness and market competitiveness. While this is easily understood, it is perhaps surprising that little effort has been made so far to specify business rules rigorously and explicitly during system development. System analysts have long been able to model an organisation in terms of the data structures it uses and the functions it performs. These data structures and functions represent certain business rules within an organisation, and are usually documented in various analytical and design documents — but there are many other business rules which are not articulated until it is time to convert them into program code [1]. Techniques to support the modelling of business rules explicitly during system development have begun to emerge only recently [3].

The lack of any explicit modelling of business rules during system development in the past has left many organisations facing two major problems today. Firstly, there is a problem of ‘inconsistent enforcement of business rules’. Typically, the first implementation of business rules in information systems synchronises well with the intended policies. Over time, however, the implemented business rules tend to deviate from the intended ones. This is because as the systems grow and age, knowledge about the implemented business rules gradually tends to get lost. As a result, subsequent maintenance work on the implemented business rules may not be carried out correctly and may introduce inconsistency, i.e. the business rules enforced by the supporting information systems do not necessarily correspond to the business rules that the organisation believes to be in place. Consequently, many organisations find that their legacy information systems are inhibiting their business growth and capacity to change.

Secondly, there is a problem of ‘misunderstanding of business information’. Many organisations now face the problem that their current information systems are legacy systems operating to business rules that are often unknown

to the users of the systems. This implies that the data delivered by such systems is often out-of-context.

Consider, for example, a data warehouse which integrates customer information from a number of databases, each operating to its own business rules that determine whether a customer is inactive. A user who is unaware of these rules may not be able to distinguish different types of inactive customer when retrieving customer information from the data warehouse. Consequently, vital business data can be used out-of-context (e.g. product advertisements may be sent to a customer who is marked active, but only because of uncleared debt), and time may be wasted in searching for the data that is actually relevant and useful (e.g. additional querying of customer data may be required to find the targeted active customers).

Many organisations have substantial investment in undocumented business rules implemented in the supporting information systems and are becoming increasingly reliant on these systems to support business practices.

The ability to recover business rules from information systems and make them explicitly available to system users is therefore important and beneficial. It helps to:

- enhance data quality — to understand data accurately (i.e. why a customer is or is not marked inactive), it is essential that the relevant business rules are understood completely; with the business rules explicitly available, users can truly consume valuable business information, rather than the out-of-context data,
- evolve business rules — evolving business rules is not an easy task, as it is often the case that the business rules encoded in the information systems become lost due to a variety of circumstances, such as departure of key personnel, but, by making business rules explicitly available, it will be easier for organisations to evolve or develop new rules by modifying the existing rules,
- support system maintenance — the availability of business rules will allow system engineers to have a better understanding of system functionality, hence making maintenance easier.

This paper gives details of studies into the types of business rules that may be found in an organisation's information systems, and the techniques that may be used to recover business rules from such systems. A new approach is then outlined which may be used to unlock business rules from legacy information systems.

2. Categories of business rules

The term business rule is often referenced with different meanings. In this paper, the definition given in Hay and Healy [1] is followed:

‘A business rule is a statement that defines or constrains some aspect of the business. It is intended to assert business structure or to control or influence the behaviour of the business.’

- **Example 1** Consider the following statements in English.

— Anyone placing an order with us is a customer.

— Customers are identified by address, postcode and contact.

— Only managers can discount merchandise.

— Past due accounts should be displayed in red in the interface.

These are all business rules which are derived from business requirements.

It should be noted that not all business rules are implemented in information systems. Some, such as certain legal and regulatory rules, may be enforced by the business environment. This paper only considers the business rules that are implemented in information systems.

The following sections provide a study of different types of business rule. An information system (IS) is defined to be:

$$IS = \langle D, E, C \rangle$$

where D is a set of data objects, E is a set of events and C is a set of constraints. The set of objects D , such as customers and orders, define the information structure for IS . It contains the persistent information that IS can handle. The set of events E , such as ‘when a customer places an order, an order number is created’, defines largely the functions of IS . These functions may result in state changes for D . The set of constraints C specify the valid states for D , thus limiting what IS can do to D with E . The way in which D , E and C are defined within IS is determined by business requirements. Business rules are therefore classified accordingly into three groups — structural, behavioural and constraint rules.

2.1 Structural business rules

Structural business rules are statements about the data objects with which an organisation's business deals. Different levels of description of data objects are possible.

At the lowest level, any definition of business terms is a structural business rule.

- **Example 2** *Consider the following statement.*

— A telephone line is an asset.

This statement gives one definition of what a telephone line is. This is considered as a structural business rule because it defines one of the business requirements, i.e. a telephone line is an asset, not merely equipment.

Currently this type of business rule is typically understood by human experts and not implemented as part of an information system. The recent work on ontology is trying to support explicit modelling of such knowledge and its implementation alongside the objects themselves within the systems.

Defining the terms means that structural business rules can relate terms to each other.

- **Example 3** *Consider the following statements.*

— Customers are identified by their address, postcode and contact.

— A customer places an order.

— A customer is a person.

— A car is made of wheels and a body.

These statements are structural business rules as they define the relationships between the various data objects. It should be noted that these relationships are derived from business requirements.

Such structural business rules are usually inherent within a data model and are implemented as part of a database system. Relatively complex business logic can be defined implicitly through a good data model.

An information system usually distinguishes the stored objects from the derived ones. A stored object is physically stored within the system, whereas a derived one is created by an inference or a mathematical calculation using the stored objects. Such derivations are regarded as structural business rules too, because, from the end-user point of view, the derived objects are not different from the stored ones.

- **Example 4** *The following derivations are structural business rules.*

— The insurance amount in rental is calculated from the insurance rate multiplied by the number of days.

— The total cost of the rental is calculated from the sum of insurance amount and rental amount.

Here, whether the total cost of rental is a stored entity or a derived one is irrelevant to an end user. The only important thing is that the system should deliver a correct total. This is why such derivations are considered as structural business rules. However, from the business and system maintenance point of view, it is useful that the actual derivation is understood explicitly and separately. It should be noted that these business rules are usually implemented as program code.

2.2 Behavioural business rules

Behavioural business rules are statements about some dynamic aspects of a business. They specify what can or will be done to data objects in response to events, and the actions taken may result in change of state for the data objects.

- **Example 5** *The following statements are examples of behavioural business rules.*

— When an insurance claim is made, a claim reference number is allocated.

— If a customer withdraws money, the corresponding account is debited.

These rules specify what the system will do in response to an event.

Behavioural business rules often implement much more complex business logic than can be defined through a data model, and they are usually implemented as procedural programs.

2.3 Constraint business rules

Constraint business rules are statements about the conditions under which an organisation's business operates. They express the valid states of data objects and ensure that only those actions which result in valid states for data objects may be performed.

- **Example 6** *The following are constraint business rules.*

— Customers cannot withdraw money if their accounts are overdrawn.

— A customer must have at least one account, but no more than three.

— A date is mandatory for each transaction.

— Active accounts must have customers.

— Discounts on merchandise can be set by managers.

Constraint business rules may be implemented in various parts of an information system. The last constraint in Example 6, for instance, may be implemented as part of the authorisation sub-system. As mentioned in section 1, system analysts did not, in the past, model such rules formally at analysis and/or design stages. Furthermore, in older systems constraint business rules tend to be implemented as program code.

2.4 Summary

This section closes with the following observations on business rules [1].

- Firstly, a business rule is declarative, not procedural. It describes a desirable or possible state that is either required or prohibited. It may be conditional, i.e. if something is the case, something else must or must not be the case. It does not, however, describe the steps to be taken to achieve the transition from one state to another, or the steps to be taken to prohibit a transition.
- Secondly, a business rule may or may not be stated explicitly in an organisation. For example, the last business rule in Example 1 is typically implemented as part of a system, but not formally stated anywhere else. The unstated business rules are equally important. For the purposes of this study, they are even more important than the stated ones. It is this type of business rule that gradually gets lost in large information systems over time.
- Thirdly, different types of business rule may be implemented in different ways, in different parts of a system and require different techniques to recover them. For example, structural business rules are more likely to be implemented as part of a data model, whereas constraint business rules may be found in procedural code. A full understanding of different types of business rule is therefore important.
- Finally, business rules can be classified roughly into three groups — structural, behavioural and constraint rules. As pointed out in section 1, the business rules that define business practices are better understood and documented compared to the business rules that constrain business practices. Consequently, it is perhaps more important to recover constraint business rules. The next section covers the techniques that may be used to recover business rules from legacy information systems.

3. Extracting business rules

The study of business rule extraction from legacy information systems is closely related to the study of software reverse engineering. There is a rich collection of techniques that have been developed as a result of many years of

research in this field. This section provides a high-level summary of these techniques by classifying them into two broad groups — ‘data-understanding techniques’ and ‘program-understanding techniques’ — and comments on their suitability for extracting business rules.

A software reverse engineering task is largely characterised by its input and output. The output depends on the reason for performing reverse engineering, whereas the input depends on the available resources. The inputs and outputs vary widely from task to task, and are major factors that influence the appropriate reverse engineering approach or technique [4]. The following discussion tries to identify possible input and output for data-understanding and program-understanding techniques, respectively.

3.1 Data understanding

Data-understanding techniques have been developed mainly by the database research community. Their main objectives are to recover conceptual data models from databases [5, 6], and to use the recovered models to support activities such as database integration, migration or re-engineering.

The inputs for data understanding vary. The possible inputs can be classified into four main groups.

- Schema

The schema is normally the dominant input for data understanding. It gives structural and some constraint information about data explicitly. Clearly, the availability of schema information can greatly help data understanding, but the variability in DBMS, design style, etc, means that care must still be taken when trying to recover data models from schema information.

- Data

The data stored in a database may also be examined for data understanding purposes. This is because data may contain certain properties that are not specified in the schema, e.g. certain functional dependencies. Data is therefore a useful supplementary input for data understanding.

- Programs

Program code associated with a database system is another type of input. A database system may include application programs, stored procedures or user interface forms. As these programs operate on data objects, they often contain clues about data structures and constraints. These clues, when used in conjunction with schema/data, could potentially help data understanding.

- Transactions

Typically, operations against a database are recorded in a transaction log in a database system. This transaction log may be used as an input for data understanding too. By analysing the way in which the database is used (the recorded transactions), it may be possible to derive a data model that is perceived by the users.

Two observations are made on the possible inputs for data understanding.

- Human intervention

It is widely recognised that complete automation of data understanding is not possible, and human intervention will be necessary. Such human intervention may also be regarded as a form of input, but, for the purposes of this paper, such input is ignored in this classification.

- Schema/data input

Most of the current data-understanding techniques work with schema input [7]. The recent advances in data mining have prompted the use of data as input too [8], but it is still very limited. The need to analyse programs in understanding data is beginning to be recognised [3] and still needs more research effort (see section 4 for a possible approach). Finally, the use of transaction logs in data understanding, though promising, has yet to be explored.

The output from data understanding is usually a data model which describes data conceptually. A complete conceptual data model should include not only the data structures, but also the constraints. Therefore, using schema information only, as currently practised, may not be sufficient for recovering a complete conceptual data model. Occasionally, a mapping between data models is required as the output of data understanding [4]. However, the generation of model mapping is considered to be a forward-engineering activity, following the reverse engineering of respective data models first.

Clearly, data-understanding techniques are useful in recovering business rules (in particular, structural business rules) that are buried in the database and its metadata, but they typically do not analyse the application programs, which also contain business rules. For example, business rules governing database updates are usually part of an application program, which would not be analysed by a data-understanding technique. A new approach is therefore required (see section 4).

3.2 Program understanding

Program-understanding techniques have been developed mainly by the software engineering research community. Their main objectives are to recover design from programs [9], and to use the recovered design to support activities, such as software maintenance, reuse or quality assurance.

In contrast to inputs for data understanding, inputs for program understanding are relatively straightforward — mainly the program source code (here again, human intervention is ignored). A source program is usually translated into an abstract syntax tree representation before it is analysed. Therefore, the assumption that program-understanding techniques work on this generic representation of source programs would mean that effectively there is one type of input.

The output from program understanding is, however, less uniform and captures the complexity of the task involved. The following forms of outcome from a program-understanding task are shown in Fig 1.

	syntactic	semantic
component		
architecture		

Fig 1 Outcome from a program-understanding task.

The great majority of the current techniques aim to understand the components of a program syntactically. These techniques analyse source code, produce useful information such as call graphs, and compute metrics such as coupling and cohesion. Once the components are identified from usually large and monolithic programs, they may be reused or re-engineered without affecting other parts of the programs. Such techniques are well on their way to maturity. Tools, such as Reasoning Systems' Software Refinery [10], alleviate most practical problems in discovering program component structures.

More recently, techniques to recover system architectures, syntactically, from source code have started to appear in the literature. Recovering architectural features from source code is also labelled as design recovery. With some human assistance, these techniques can recover various high-level design constructs from the source code, e.g. control flows or design rationale. The recovered designs offer a higher level of abstraction or understanding of the source code compared to that which can be achieved by understanding the components.

Syntactic analysis of programs does not take account of the meaning of a program. Therefore, there is a fundamental upper limit to what can be achieved in syntactic program understanding. Increasingly, there is a growing interest in trying to understand a program semantically.

To understand program components semantically, most of the work relies on a knowledge-based approach. Using this approach, program plans or clichés are designed and stored in a knowledge base. If a particular program fragment or component matches one of the plans stored in the knowledge base, the meaning of this fragment or component is deemed to be 'understood'. Clearly, this level of program understanding is essential for extracting business rules from programs. However, semantic understanding of program components is still at an early stage and very few tools exist currently.

Finally, semantic understanding of software architecture is included for completeness. In this case, it is hoped that the purpose for which the software is designed can be recovered from the code, i.e. by building on the components that have been understood and deriving a higher level of understanding of the software semantically. In so doing, a domain is usually constructed to provide the required context and guidance for understanding programs this way. Generally speaking, this is very ambitious and requires heavy human assistance. Luckily, this level of understanding is not required to extract business rules from information systems.

In summary, the program-understanding techniques are useful in locating relevant program fragments and identifying their relationships, but typically they do not analyse the fragments in relation to database systems. For example, program-understanding techniques can be used to identify the section of code which carries out a database update, but they would be unable to analyse the way in which the update was propagated through the database by triggers stored in the database system. Thus, these techniques are most useful in recovering business rules from systems that consist of programs only. For many business applications, however, data plays a dominant role, and therefore new approaches will be required.

4. The way forward

It is obviously not possible to recover all types of business rule from all types of information system using a single technique. In this section, therefore, a data-centred program-understanding approach is suggested for unlocking one particular type of business rule (constraint business rules) from one particular type of system (systems consisting of databases and programs), i.e. databases and programs will be analysed together to recover constraint business rules that are scattered across application programs, data dictionaries, triggers and stored procedures.

As a first step in unlocking business rules from legacy systems, it is felt that this focus is important for the following reasons.

- Systems analysts have long been able to model an organisation in terms of the data structures it uses and the functions it performs. These data structures and functions represent certain structural and behavioural business rules and are usually documented in various analytical and design documents. What systems analysts tend to neglect during system development are the constraints under which an organisation operates. Frequently, these constraints are not articulated until it is time to convert them into program code [1]. Thus, unlocking constraint business rules is perhaps the area of greatest need.
- Constraint business rules are of crucial importance in decision support systems [11]. Such systems typically require constraint business rules to be distributed between database systems and application programs. Consider, for example, the entry of data into a data warehouse. Rules governing the transformation of data prior to entry are typically implemented as application programs, while rules dealing with the propagation of new data (e.g. in updating aggregated data) and validation (e.g. in enforcing conformance to a standard) are more usually implemented using triggers and stored procedures in the database systems. To analyse the constraint business rules in such systems, data-understanding and program-understanding techniques in isolation are not sufficient. Databases and programs must be analysed together.

So far, little work has been done in addressing the particular problem outlined here [3, 12], and it is believed that the proposed research has the potential to make significant contributions to this area of study.

The approach as outlined in Fig 2 consists of four components — parsing tools, schema tools, extraction tools and presentation tools. The whole process is seen as consisting of three stages — preparation, extraction and presentation. The system will have a basic user interface to control its operations.

The preparation stage translates the source system, both program code and databases, into a generic form. 'Parsing tools' are used to convert a program into an abstract syntax tree representation. This representation is then stored in 'program base'. It is possible to work on source programs directly, but the benefit of firstly converting them into a generic representation is that this system will be extensible and can analyse different source programs in different languages together [13]. Schema tools are used to derive conceptual data models (CMs) from data or databases. These tools are not difficult to develop [14, 15].

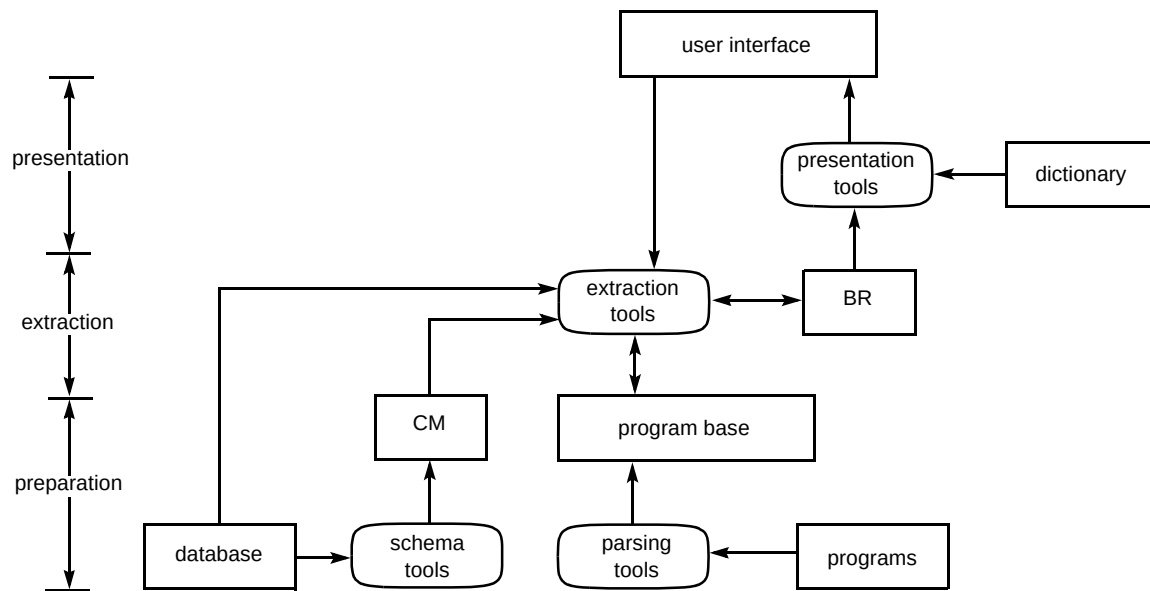


Fig 2 The data-centred program-understanding approach.

The extraction stage extracts business rules from the generic representation of programs and databases. Extraction tools are used to recover constraint business rules from application programs, triggers and stored procedures which have all been imported into the 'program base'. The discovered rules are stored in BR. Unlike conventional program understanding, the analysis of the programs in 'program base' will be centred around the schema information contained in the CM, i.e. the available schema information will be used as a guide to analyse the programs. The development of extraction tools is divided into two tasks. The first task is to develop a mechanism for expressing extraction 'guidance'. The extraction technique is expected to be heuristic and closely guided by the schema information. Therefore, it is essential that the relevant heuristics and schema information can be expressed in such a way that they can be used by this extraction technique as a guide. There is some existing work which uses program plans and a program algebra to express certain program patterns in program understanding [13, 16]. These methods will be further investigated and additional studies undertaken. The second task is to develop the extraction technique itself. The starting point will be the standard 'program-slicing technique' (including its variations) [17], which is widely and successfully used in program understanding. The aim is to extend this technique so that it:

- works with the heuristics detailed here,
- incorporates schema information (available in the CM) in its analysis.

In the presentation stage, 'presentation tools' are used to present the discovered business rules to the users. The main difficulty here is that business rules need to be presented to

different user groups in different contexts. For example, system engineers may prefer the discovered rules to be presented as source code with line numbers, whereas business users may require the same rules presented in a business context. For this reason, it is appropriate firstly to store the discovered business rules internally within the BR, and then present such rules to different users in their preferred context and format. Firstly, a simple dictionary-based approach will be used [18], i.e. users will be allowed to define their preferred terms in their dictionaries and then map them to the relevant discovered business rules. Once this succeeds, the possibility of using an ontology to further support this task will be investigated [19].

5. Conclusions

In this paper, the importance of unlocking business rules from legacy information systems has been argued. The types of business rule that may be found in an organisation's legacy information systems have been briefly described, and techniques detailed that may be used to unlock them. Analysis shows that data-understanding and program-understanding techniques are not good enough yet, when used in isolation, to unlock business rules. New approaches and more research effort will be required in order to unlock this valuable asset that many organisations have concealed in their information systems.

Also suggested in this paper is an approach to unlocking a particular type of business rule from a particular type of system — the recovery of constraint business rules that are scattered across application programs, data dictionaries, triggers and stored procedures. The work is still at an early stage and the development of a prototype system based on the approach outlined in this paper has been funded by EPSRC.

References

- 1 Hay D and Healy K A: 'Defining business rules — what are they really?', in: 'The Guide Business Rules Project' — <http://www.guide.org> (1996).
- 2 Herbst H and Myrach T: 'A repository system for business rules', in Proc of the 6th IFIP TC-2 Working Conference on Data Semantics (1996).
- 3 Henard J et al: 'Program understanding in database reverse engineering', in Proc of DEXA98, pp 70—79 (1998).
- 4 Blaha M R: 'Dimensions of database reverse engineering', in Proc of Working Conference on Reverse Engineering, pp 176—183 (1997).
- 5 Fonkam M M and Gray W A: 'An approach to eliciting the semantics of relational databases', in Proc of Entity-Relationship Approach, ER'92, pp 463—480 (1992).
- 6 Hainaut J L et al: 'Contribution to a theory of database reverse engineering', in Proc of IEEE Working Conference on Reverse Engineering (1993).
- 7 Hainaut J L et al: 'Structure elicitation in database reverse engineering', in Proc of IEEE 3rd Working Conference on Reverse Engineering, pp 131—140 (1993).
- 8 Breiteneder C J, Hits H and Mueck T A: 'Metadata mining in legacy data sets', in Proc of Metadata 96 (1996).
- 9 Chikofsky E J and Cross J H: 'Reverse engineering and design recovery: a taxonomy', IEEE Software, 7, No 1, pp 13—17 (1990).
- 10 Markosian L et al: 'Using an enabling technology to re-engineer legacy systems', Communications of the ACM, 37, No 5, pp 58—70 (1994).
- 11 Schardt J A: 'The enterprise intersection model — a unifying framework for data warehouse development', Journal of Data Warehousing, 2, No 2, pp 51—61 (1997).
- 12 Software Re-engineering Repository — <http://www.informatik.uni-stuttgart.de/ifi/ps/reengineering/>
- 13 Paul S and Prakash A: 'A query algebra for program databases', IEEE TSE, 22, No 3, pp 202—216 (1996).
- 14 Ashwell M H, Behrendt W, Gray W A and Fiddian N J: 'REVEERD — an implementation of a reverse engineering tool for relational databases', in Proc of 6th Workshop on Information Technologies and Systems, pp 81—90 (1996).
- 15 Behrendt W, Fiddian N J and Madurapperuma A P: 'Tools to enable interoperation of heterogeneous databases', in Proc of 13th International Conf on Data Engineering, p 577 (1996).
- 16 Palthehu S, Greer J E and McCalla G I: 'Cliché recognition in legacy software: a scalable, knowledge-based approach', in Proc of the 4th IEEE Working Conference on Reverse Engineering, pp 94—103 (1997).
- 17 Horwitz S, Reps T and Binkley D: 'Interprocedural slicing using dependence graphs', ACM TOPLANS, 12, No 1, pp 26—60 (1990).
- 18 DeBaud J M: 'DARE: domain-augmented re-engineering', in Proc of Working Conference on Reverse Engineering, pp 164—173 (1997).
- 19 Karunaratna D D, Gray W A and Fiddian N J: 'Establishing a knowledge base to assist integration of heterogeneous databases', in Proc of BNCO16, pp 103—118 (1998).



Dr Jianhua Shao received his BSc degree from Shanghai University in 1982, MPhil from CNAA in 1986 and PhD from University of Ulster in 1990, all in computer science. He joined Cardiff University in 1996 as a lecturer in computer science. Prior to that, he held research and lecturing posts at the University of Ulster. His research interest is broadly in the area of database systems. Specific interests include data mining, database semantics and business rules in legacy information systems. He has over 20 research publications.



Chris Pound joined BT in 1985 after receiving a BSc degree in Geology from the University of Wales (Swansea) and a period of research at the University of Plymouth where he received his PhD in sedimentology. His career in systems engineering at BT has encompassed a range of roles in systems analysis, requirements management, solution design and project management across a wide range of business areas. Specialist roles in the area of information management account for two-thirds of this career span.

In 1999 he joined BTUK Strategy where he currently holds responsibility for Information and Knowledge Management Strategy.